Synopsis:

“Air Quality Level Classification System”

Problem Id: 19

Student Name: Anshul Thapliyal(2028646)

Student Name: Pranav Sharma(2028800)

Student Name:Riya(2028824)

Section : L

• Problem Description:

In today's rapidly urbanizing world, air pollution has become a serious environmental and public health concern. Rising levels of pollutants from vehicles, industries, and other human activities have made it essential to monitor and analyze air quality effectively. Traditional methods of analyzing air quality data are often time-consuming and inefficient due to the large volume of data generated by environmental sensors.

This project focuses on building an air quality classification system, where environmental conditions are analyzed using sensor data such as Carbon Monoxide (CO), Nitrogen Dioxide (NO_2), temperature, humidity, and particulate matter levels. By identifying patterns in this data, the system can classify air quality into different categories, enabling better understanding and decision-making.

In Phase 1, the theoretical design of the machine learning pipeline was completed. Concepts such as data preprocessing, feature scaling, feature selection, and classification algorithms were studied.

In Phase 2, the dataset was collected and preprocessed. Missing values were handled, noise and outliers were removed using statistical methods like Z-score, categorical variables were encoded, and numerical features were scaled to ensure consistency.

In Phase 3, machine learning algorithms such as K-Nearest Neighbors (KNN) and Logistic Regression were applied to classify air quality levels. The dataset was split into training and testing sets, and models were trained and evaluated using performance metrics like accuracy, precision, and confusion matrix.

This project helps in understanding air pollution patterns and provides a reliable system for classifying air quality, which can assist authorities and individuals in taking preventive and corrective measures.

➔Dataset Description

The dataset contains multiple environmental features such as:

- Pollutants: PM2.5, PM10, NO_2, CO, SO_2, O_3
- Environmental factors: Temperature, Humidity
- Target variable: **AQI_Bucket (Air Quality Category)**

Categories include:

- Good
- Satisfactory
- Moderate
- Poor
- Very Poor
- Severe

- **Exploratory Data Analysis (EDA):**

1. Used head(), info(), describe() to understand data
2. Identified missing values and inconsistencies

Steps of Implementation

- **Data Preprocessing**
  - ➔ **Handling Missing Values**
    - Numerical → Mean/Median
    - Categorical → Mode

➔ **Outlier Removal**

Using Z-score:

Z = (x-u) / o (u is mean, o is standard deviation)

➔ **Feature Scaling**

Standardization:

X-Standard : Xstd = (X-u) / o (u is mean, o is standard deviation)

➔ **Encoding**

- AQI categories encoded for model compatibility

➔ **Output Files**

- X_final.csv → Features
- y_final.csv → Target

➔ Data Splitting

Code:

```
X_train, X_test, y_train, y_test = train_test_split(   X, y, test_size=0.2, random_state=42)
```

- Training Data → 80%
- Testing Data → 20%

● Model Development

1. K-Nearest Neighbors (KNN)

Code:

```
knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)
```

- Uses distance-based classification
- Works well with scaled data

2. Logistic Regression

Code:

```
lr = LogisticRegression(max_iter=1000)

lr.fit(X_train, y_train)
```

- Linear classification model
- Provides probabilistic output

● Model Evaluation

➔ Accuracy

Formula

Accuracy=CorrectPredictionsTotalPredictionsAccuracy = \frac{Correct Predictions}{Total Predictions}Accuracy=TotalPredictionsCorrectPredictions

➔ Precision

Formula

Precision=TPTP+FPPrecision = \frac{TP}{TP + FP}Precision=TP+FPTP

➔ Confusion Matrix

- Shows correct vs incorrect predictions

➔ Classification Report

Includes:

- Precision
- Recall
- F1-score

Code:

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Output Analysis

➔ **KNN Results:**

- Performs well on non-linear data
- Sensitive to scaling

➔ **Logistic Regression Results:**

- More stable
- Works better when data is linearly separable

➔ **Comparison:**

- KNN may give higher accuracy with proper tuning
- Logistic Regression is faster and simpler

- ## Visualization

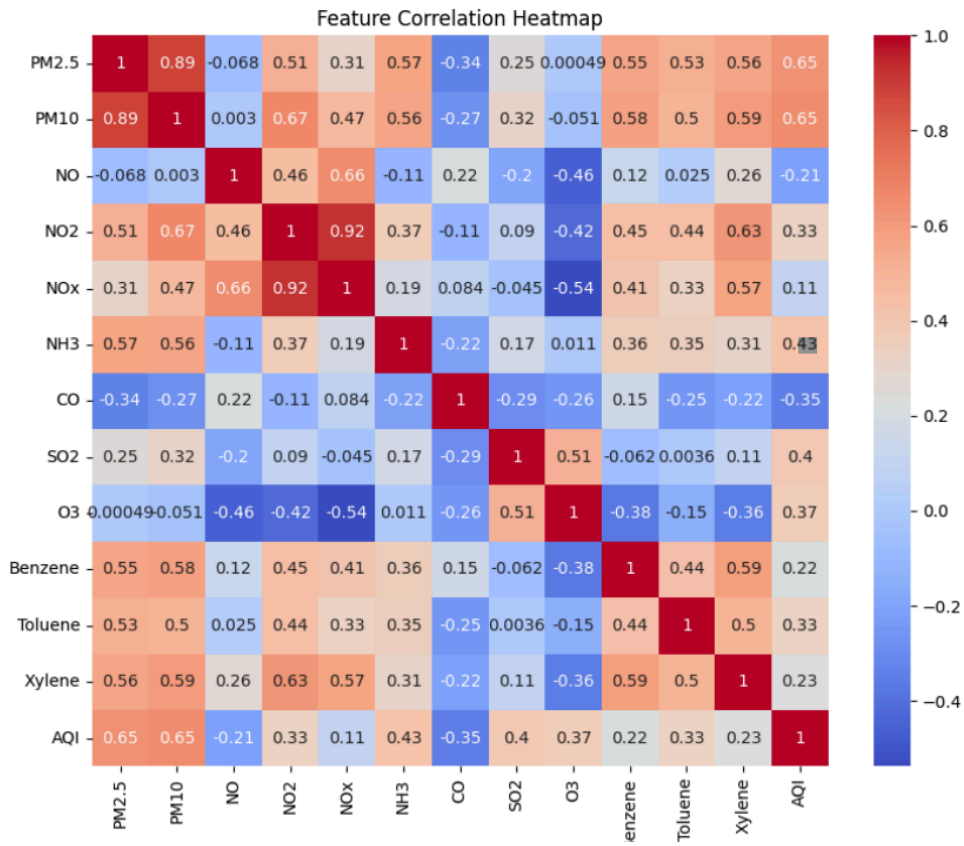
➔ **Correlation Heatmap**

This plot shows the relationship between different features. It helps in identifying important variables and removing redundant ones.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,8))
sns.heatmap(X.corr(), annot=True, cmap='coolwarm')
plt.title("Feature Correlation Heatmap")
plt.show()
```

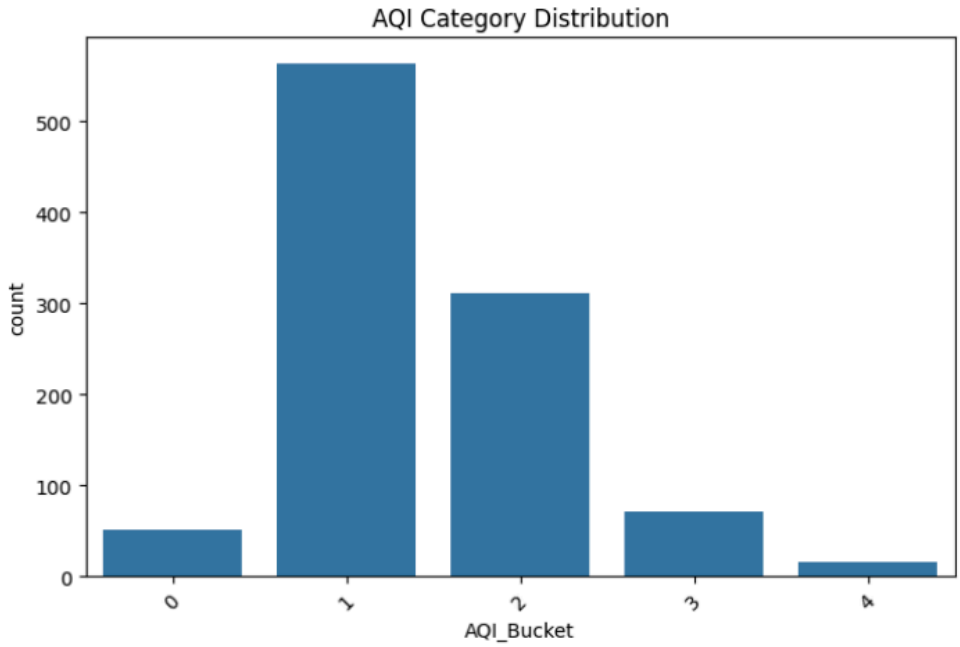


➔ **AQI Category Distribution**

This plot shows how many data points belong to each air quality category. It helps in checking whether the dataset is balanced.

Code:

```
plt.figure(figsize=(8,5))
sns.countplot(x=y)
plt.title("AQI Category Distribution")
plt.xticks(rotation=45)
plt.show()
```



➔ **Confusion Matrix (KNN)**

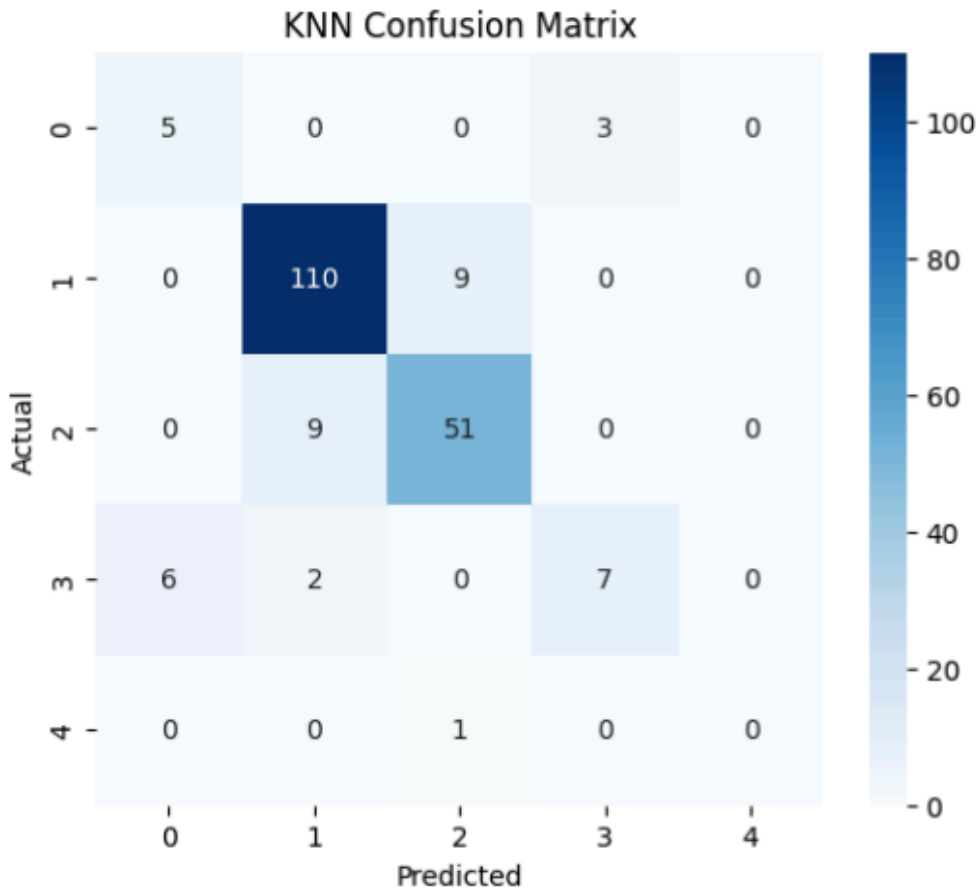
This matrix shows the performance of the KNN model by comparing actual and predicted values.

Code:

```
from sklearn.metrics import confusion_matrix

cm_knn = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(6,5))
sns.heatmap(cm_knn, annot=True, fmt='d', cmap='Blues')
plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



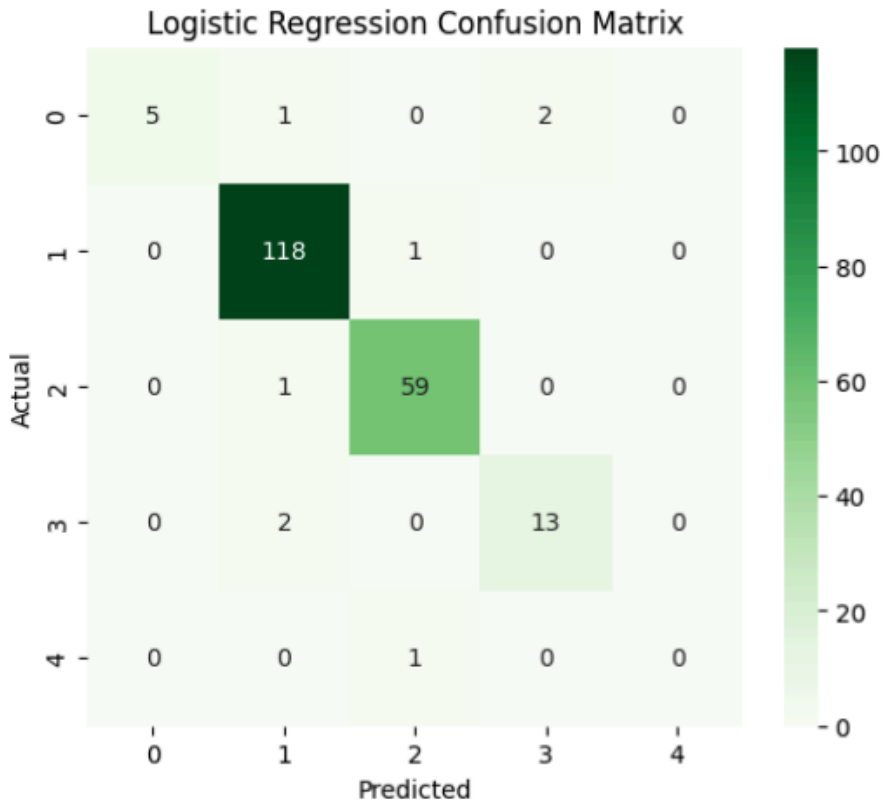
➔ **Confusion Matrix (Logistic Regression)**

This matrix evaluates the performance of the Logistic Regression model.

Code:

```
cm_lr = confusion_matrix(y_test, y_pred_lr)

plt.figure(figsize=(6,5))
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Greens')
plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



➔ **Accuracy Comparison Graph**

This bar graph compares the accuracy of KNN and Logistic Regression models.

Code:

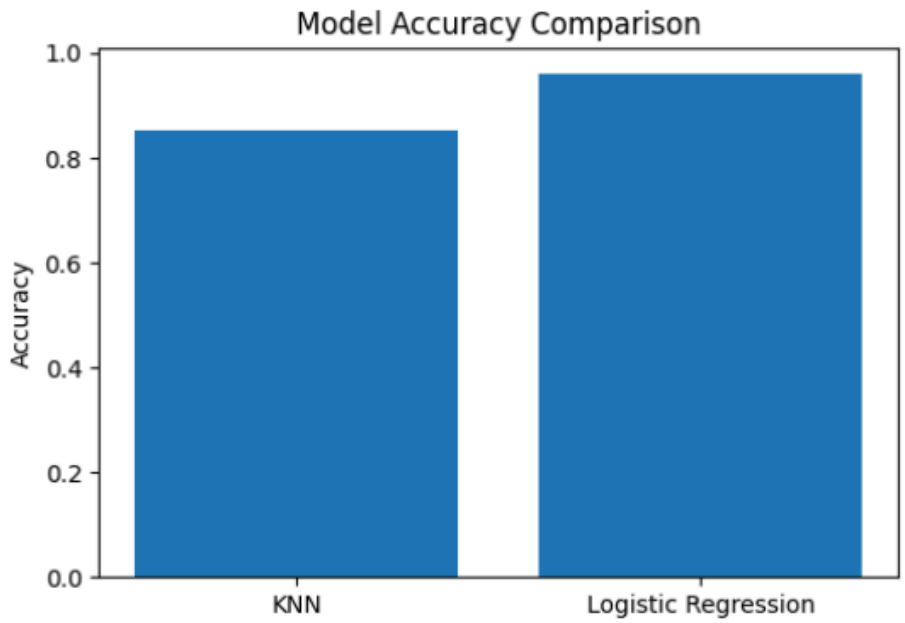
```
from sklearn.metrics import accuracy_score

knn_acc = accuracy_score(y_test, y_pred_knn)
lr_acc = accuracy_score(y_test, y_pred_lr)

models = ['KNN', 'Logistic Regression']
accuracies = [knn_acc, lr_acc]

plt.figure(figsize=(6,4))
plt.bar(models, accuracies)
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
```

plt.show()



- ## Output Analysis

The models were evaluated using accuracy, confusion matrix, and classification report.

KNN Model performs well on non-linear data and gives good accuracy after scaling.

Logistic Regression provides stable and consistent results.

From the **confusion matrix**, most values lie on the diagonal, indicating correct predictions, while some misclassifications occur between similar categories like *Moderate* and *Poor*.

The **classification report** shows good precision, recall, and F1-score for most classes, with slightly lower performance for less frequent categories.

- · **Expected Modules/ Libraries to be used :**

1. NumPy:

It is used to perform numerical calculations and work with arrays efficiently, which makes handling large amounts of data easier.

2. Pandas:

It helps in organizing and managing the dataset, making it simple to read, clean, and analyze data.

3. Matplotlib:

It is used to create basic graphs and charts, helping us visualize the data in a clear way.

4.Seaborn:

It provides more advanced and attractive visualizations, such as heatmaps, to better understand relationships in the data.

5. Scikit-learn (sklearn):

It is the main library used to build machine learning models like KNN and Logistic Regression, and also to evaluate their performance using different metrics.

- · **Expected Evaluation metrics and Plots to be used:**

1. Accuracy: Measures overall correctness of the model.

2.Precision: Measures correctness of positive predictions.

3.Confusion Matrix: Shows correct and incorrect classifications.

4. Heatmap: Visual representation of correlation between features.

5. Bar Graphs: Used to compare model performance.

These metrics and plots help in understanding model performance and selecting the best model.

- · **Required topics from the subject:**

1. Data Structures (Arrays):

Arrays are used to store and manage the dataset values in an organized way, making it easier to process and perform operations on the data.

2. File Handling:

File handling is used to read the dataset (usually in CSV format) so that it can be used in the program for further processing.

3. Functions:

Functions help in breaking the code into smaller, manageable parts, which improves readability and allows code reuse.

4. Basic Python Programming:

Basic Python concepts are used to implement the overall logic of the project and connect all the steps together smoothly.

- **Platform Required:**

1. Jupyter Notebook
2. Python IDLE
3. Visual Studio Code

- · **Books and Link Sources:**

1. "Hands-On Machine Learning with Scikit-Learn" by Aurélien Géron
2. Python for Data Analysis" by Wes McKinney
3. Kaggle (https://www.kaggle.com/□)
4. Scikit-learn Documentation (https://scikit-learn.org/□)